

CashShield Document



CashShield v2

Probabilistic Financial Resilience Simulator

Product Requirements Document (PRD) — Version 2

Document Metadata

- **Version:** v2.0
 - **Status:** Functional prototype (Streamlit-based)
 - **Last Updated:** February 2026
 - **Author:** MD Tanvir Anjum
 - **Evolution From:** CashShield v1 (Local LLM, conceptual-first prototype)
-

Executive Summary

CashShield v2 is an interactive financial risk simulation tool that enables users to **quantitatively measure their probability of cash insolvency under realistic uncertainty**, while receiving **clear, human-readable explanations** of *why* risk exists and *which interventions matter most*.

The Core Problem: Traditional financial tools optimize for wealth accumulation, but most people's immediate concern is **survival** — not running out of money during uncertain times. Existing solutions fail because they rely on average-case thinking, ignore timing risk, and provide prescriptive advice rather than empowering decision-making.

The Solution: CashShield combines Monte Carlo simulation with AI-powered explanations to help users understand their financial fragility and make informed decisions about risk mitigation.

Why This Matters: Financial stress is one of the leading causes of anxiety and poor life outcomes. Yet most people operate with incomplete mental models of their financial situation. CashShield bridges this gap by making uncertainty visible, actionable, and understandable.

1. Problem Statement

1.1 The Core Problem

People systematically underestimate their financial fragility because they think in averages, not distributions.

Why This Happens:

Cognitive Biases:

- **Availability heuristic:** People plan based on recent, smooth financial experiences, ignoring potential disruptions
- **Planning fallacy:** Systematic underestimation of how long problems take to resolve (e.g., finding a new job)
- **Base rate neglect:** Personal finance feels unique, leading people to ignore statistical realities of income volatility

Tool Limitations:

- **Budgeting apps** show current spending but don't model future uncertainty
- **Retirement calculators** assume smooth growth curves with no life disruptions
- **Bank balance** is a single data point that reveals nothing about resilience
- **Financial advisors** focus on investment returns, not survival probability

Information Asymmetry:

- Financial institutions profit from complexity and opacity
- Probabilistic thinking is cognitively demanding and poorly taught
- Most people lack frameworks to reason about uncertainty in personal contexts

Real-World Manifestation:

A person sees:

- "I have £5,000 in savings"
- "My income usually covers expenses"

- "Therefore, I'm fine"

They don't see:

- "If I lose my job and it takes 4 months to find another, I'll run out of money in month 3"
- "If a major expense hits in month 2, my runway drops to 6 weeks"
- "My income varies by $\pm£400$ /month, which means 1-in-4 months are structurally negative"

This is a reasoning gap, not an information gap.

1.2 Why Existing Solutions Fail

Solution Type	What It Does	Why It Fails	Evidence/Example
Budgeting apps (YNAB, Mint)	Track past spending	Backward-looking; no uncertainty modeling	User thinks "I'm under budget" but one car repair breaks them
Retirement calculators	Project wealth growth	Assumes smooth compounding; ignores shocks	2008 financial crisis invalidated a decade of "safe" projections
Financial advisors	Recommend investments	Expensive; return-focused, not risk-focused	Advisors rarely model "what if you lose your job for 6 months?"
Spreadsheets	Manual forecasting	Single-path thinking; no distribution awareness	Users model "expected case" which is rarely realized
Emergency fund rules	"Save 3-6 months expenses"	Generic; doesn't account for income volatility or individual risk factors	3 months might be catastrophic for gig workers, excessive for stable jobs

The Gap: No consumer tool bridges the space between "here's what happened" (budgeting) and "here's what's likely to happen under realistic uncertainty" (probabilistic simulation).

1.3 Why This Problem Matters

Individual Impact:

- Financial stress is the #1 source of anxiety in developed countries

- Inability to assess true risk leads to either over-caution (foregone opportunities) or under-caution (preventable crises)
- Most personal bankruptcies result from unexpected events + thin buffers — both are modelable

Systemic Impact:

- Financial fragility creates social instability
- Prevention is vastly cheaper than crisis intervention
- Better individual risk awareness could reduce demand for predatory lending

Timing: This problem is especially acute now because:

- Gig economy increases income volatility
 - Traditional employment offers less security
 - Cost of living shocks (rent, energy) are more frequent
 - Economic uncertainty is high (inflation, recession fears)
-

2. Why v2 Was Necessary

2.1 Core Insight From v1

CashShield v1 successfully validated three hypotheses:

1. Monte Carlo simulation meaningfully reveals hidden financial fragility

- Users consistently reported "I had no idea my situation was this precarious"
- Gap between perceived and actual risk was largest for those with medium savings (£3K-£10K)

2. Users strongly value *why* explanations, not just risk scores

- Users spent 3x longer reading explanations than viewing visualizations
- Follow-up questions consistently asked "why is my fixed-cost ratio a problem?"
- Quantitative scores without context created anxiety, not clarity

3. Average-based thinking is deeply misleading in personal finance

- Users whose "average month" looked fine were shocked to see 30-40% failure rates
- Volatility + timing effects were completely invisible in traditional planning

Validation Signal: 8/10 prototype testers said "I would pay for this" despite it being rough and local-only.

2.2 Critical Limitations of v1

Through user testing and technical evaluation, five structural problems emerged:

Limitation #1: Deployment Constraints

The Problem:

- v1 used Ollama (local LLM), requiring users to install and run models locally
- This was acceptable for technical users but created a massive barrier for mainstream adoption
- Streamlit Cloud (target deployment) doesn't support local LLMs

Why This Mattered:

- Limited audience to technically sophisticated users
- Prevented rapid iteration based on user feedback
- Made sharing/demonstrating the tool difficult

The Deeper Issue: Architecture tightly coupled AI to infrastructure, creating deployment fragility.

Limitation #2: Cognitive Overload on First Use

The Problem:

- v1 led with explanation and conceptual depth
- First-time users were overwhelmed before understanding if the tool was relevant to them
- Information architecture assumed users wanted to learn methodology before seeing results

User Feedback (Actual Quotes):

- "This is fascinating, but I don't know what I'm supposed to do"
- "Can you just tell me if I'm okay first?"
- "I got lost in the explanation before understanding my risk"

Why This Mattered:

- High cognitive load → high abandonment
- Tool felt like homework, not decision support
- Users wanted **triage first, depth second**

The Deeper Issue: Product was designed for how I think (analysis-first), not how users experience financial anxiety (threat-first).

Limitation #3: Implicit Scenario Testing

The Problem:

- v1 showed current risk but didn't make it easy to test changes
- Users had to mentally simulate "what if I reduced expenses by £200?"
- No clear way to compare before/after states

User Feedback:

- "This is scary — what would actually help?"
- "Can you show me what happens if I save more?"
- "How much do I need to change to make a difference?"

Why This Mattered:

- Risk awareness without actionable paths creates anxiety, not empowerment
- Users want decision support, not just diagnosis

The Deeper Issue: Tool was a **diagnostic** when it needed to be a **decision lab**.

Limitation #4: AI Authority Ambiguity

The Problem:

- In v1, LLM outputs were intertwined with simulation results

- Users sometimes couldn't tell what was calculated vs. interpreted
- Risk of over-trusting AI "recommendations" when the value is in the simulation

Why This Mattered:

- Violates responsible AI principles (transparency, appropriate reliance)
- Created liability concerns if users made decisions based on AI suggestions
- Undermined trust in the simulation engine (the actually reliable part)

The Deeper Issue: Unclear separation between **authoritative calculation** and **interpretive explanation**.

Limitation #5: Conceptual Flexibility Without Guardrails

The Problem:

- v1 allowed arbitrary inputs with minimal constraints
- This was intellectually satisfying but overwhelming for real users
- Users didn't know what "realistic" volatility values were

User Feedback:

- "I have no idea what to put for income volatility"
- "Is 15% high or low?"
- "Can you just set defaults that make sense?"

Why This Mattered:

- Garbage in, garbage out — unrealistic inputs produce misleading results
- Users wanted smart defaults, not blank slates
- Flexibility that requires expertise isn't actually flexible

The Deeper Issue: Product optimized for **power users** when target was **uncertain but capable users**.

2.3 Design Response: v2 Architecture

v1 Limitation	Root Cause	v2 Solution	Why This Works
Local-only LLM	Tight coupling of AI to infrastructure	Modular AI layer (local OR cloud via	Separates concerns; enables deployment

v1 Limitation	Root Cause	v2 Solution	Why This Works
		OpenRouter)	flexibility
Narrative-heavy first experience	Analysis-first UX	Risk-first UX: headline → visual → explanation	Matches threat-processing psychology; progressive disclosure
Implicit scenario testing	Diagnostic-only design	Explicit "What-if Fix" comparison engine	Transforms insight into action; clear cause-effect
AI authority ambiguity	Intertwined logic	Hard separation: simulation = authoritative, AI = interpretive	Transparent responsibility; reduces over-reliance
Unconstrained flexibility	Power-user optimization	Structured inputs + smart defaults + validation	Lowers cognitive load; guides realistic modeling

Core Principle: v2 prioritizes **trust, clarity, and scalability** over conceptual purity.

3. Target User

3.1 Primary User Profile

Demographics:

- Age: 25-45
- Income: £25K-£75K (middle-income, not wealthy but not crisis)
- Savings: £1K-£20K (has buffer, but uncertain if it's adequate)
- Employment: Mix of stable jobs and gig/contract work

Psychographics:

- **Financially literate but uncertain:** Understands basics (budgeting, savings) but not probabilistic risk
- **Planners, not avoiders:** Actively thinks about money; wants to improve situation
- **Information-seekers:** Reads finance content; tries tools; wants to understand

- **Risk-aware but unsure:** Knows "anything can happen" but lacks framework to quantify

Key Mindset:

- "I *think* I'm okay, but I don't actually *know*"
- "I want to understand my situation, not be told what to do"
- "I'm worried but don't want to be patronized"

3.2 Why This User?

Why Not Target Crisis Users (£0 savings, immediate risk)?

- They need urgent intervention, not simulation
- Different tool category (crisis hotlines, emergency assistance)
- Simulation is valuable for prevention, not triage

Why Not Target Wealthy Users (£100K+ savings)?

- Their risk is genuinely low; simulation adds less value
- They can afford advisors; different willingness-to-pay
- Optimization (returns) matters more than survival

Why Middle-Income, Buffer-Holding Users?

- Largest addressable market
- Highest uncertainty-to-knowledge gap
- Most benefit from risk quantification
- Preventive intervention is most effective here
- Highest willingness to engage with tools

3.3 User Jobs-to-be-Done

Functional Jobs:

1. "Assess whether my current savings/income are adequate for realistic uncertainty"
2. "Understand which financial factors actually drive my risk"
3. "Test whether specific changes (e.g., saving £200/month) would meaningfully improve my situation"

4. "Get calibrated on what 'normal' financial volatility looks like"

Emotional Jobs:

1. "Reduce financial anxiety by replacing vague worry with concrete understanding"
2. "Feel in control by having a framework to reason about uncertainty"
3. "Avoid shame/judgment while exploring my situation"
4. "Gain confidence that I'm making informed decisions"

Social Jobs:

1. "Articulate my financial situation to a partner without emotion overriding facts"
2. "Make the case for lifestyle changes based on data, not fear"

3.4 User Research Insights

Finding #1: The "Comfortable but Fragile" Paradox

- Users with £5K-£10K savings *feel* secure because they're above "broke"
- But simulation reveals 25-40% failure rates with realistic shocks
- **Why This Matters:** These users have the most to gain from awareness and the most capacity to act

Finding #2: Volatility Blindness

- Users dramatically underestimate their income/expense variance
- Asked to estimate their monthly income range, users gave ±£100-£200
- Bank data showed ±£400-£800 was typical
- **Why This Matters:** Self-reporting is unreliable; tool must handle uncertainty in the uncertainty

Finding #3: "What-If" is the Killer Feature

- During testing, users spontaneously asked "what if X?" 4-5 times per session
- This wasn't about one optimal solution; it was about exploring trade-offs
- **Why This Matters:** Users want agency, not prescriptions

Finding #4: Explanation Builds Trust, Visualization Builds Urgency

- Users who saw only the risk score were skeptical ("how does it know?")
 - Users who saw trajectory charts understood risk emotionally
 - Users who read explanations trusted the methodology
 - **Why This Matters:** All three are necessary; order matters (urgency → trust → understanding)
-

4. Product Principles

These principles guide every design decision in CashShield v2:

Principle #1: Show, Don't Prescribe

What This Means:

- Reveal likely outcomes, let users decide what to do
- No "you should" language; no moral framing
- Present trade-offs, not solutions

Why This Matters:

- Respects user autonomy
- Avoids liability and ethical concerns
- Acknowledges that "right" financial decisions are context-dependent

Example Application:

-  Bad: "You should reduce your fixed costs"
-  Good: "If fixed costs decreased by 20%, failure rate would drop from 32% to 12%"

Principle #2: Resilience Over Optimization

What This Means:

- Focus on survival probability, not maximum returns
- Emphasize robustness to shocks, not perfect predictions
- Value floor protection over ceiling maximization

Why This Matters:

- Most users' primary need is avoiding catastrophe, not maximizing wealth

- Optimization requires risk tolerance; resilience acknowledges risk reality
- Aligns with how people actually experience financial stress

Example Application:

- Metric priority: Failure rate > Median ending balance
- Scenario focus: Job loss + emergency expense, not "ideal case"

Principle #3: Transparency Over Black Boxes

What This Means:

- Every result should be explainable
- Users should understand what the simulation does and doesn't account for
- Clear separation between calculation and interpretation

Why This Matters:

- Financial decisions are high-stakes; users need to trust the tool
- Opacity breeds either over-reliance or rejection
- Responsible AI requires interpretability

Example Application:

- Simulation logic is documented and visible
- AI is labeled as "interpretation," not "recommendation"
- Assumptions are stated, not hidden

Principle #4: Accessible Depth

What This Means:

- Simple surface, available depth
- Progressive disclosure: headline → detail
- No prerequisites to get value, but experts can dig deeper

Why This Matters:

- Cognitive load is the enemy of adoption
- But oversimplification erodes trust
- Users have varying needs; UX should accommodate both

Example Application:

- Risk score is immediate and clear
- Explanation is one click away, not mandatory
- Technical details (distributions, correlations) available but not front-loaded

Principle #5: Emotional Safety**What This Means:**

- Assume users are rational, circumstances are hard
- Frame risk as system fragility, not personal failure
- Provide clarity without inducing shame

Why This Matters:

- Financial stress already carries stigma
- Shame inhibits action; understanding enables action
- Ethical obligation to do no harm

Example Application:

- Language: "Your fixed costs are high relative to income" not "You're spending too much"
 - Tone: Neutral, factual, supportive
 - Never: Judgmental, prescriptive, alarmist
-

5. Core Use Case

The Primary Scenario

"If life doesn't go to plan, how often do I fail — and what would meaningfully reduce that risk?"

Detailed User Journey

Act 1: Entry (Motivation)

User State:

- Recent trigger: rent increase, friend got laid off, read scary economic news

- Emotional state: Vague worry, wants reassurance or concrete action plan
- Question: "Am I okay, or should I be worried?"

Entry Point:

- Discovers CashShield via search, social media, or referral
- Arrives at landing page or directly at tool

First Impression:

- Clear value proposition: "See how likely you are to run out of money"
- Low commitment: No account required, no payment
- Immediate engagement: Input form is short and understandable

Act 2: Simulation (Discovery)

User Action:

- Enters current financial snapshot:
 - Monthly income: £2,800
 - Fixed costs: £1,600
 - Variable expenses: £800
 - Starting savings: £6,000
- Optionally adjusts volatility (or uses smart defaults)
- Sets time horizon: 12 months
- Clicks "Run Simulation"

System Response:

- Runs 1,500 Monte Carlo simulations in ~2-3 seconds
- Calculates survival probability: 68% (32% failure rate)
- Classification: **Moderate Risk**

User Experience:

- **Immediate emotional hit:** "32% failure rate — that's much worse than I thought"

- Visual shows trajectory fan: median path stays positive, but 10th percentile dips below zero at month 5
- Realizes: "If something goes wrong in the first few months, I'm in trouble"

Act 3: Understanding (Why?)

User Action:

- Clicks "Why is my risk this high?"
- Reads AI-generated explanation

System Response (Example):

"Your 32% failure rate is driven by three factors:

1. **Thin monthly surplus:** After fixed and variable costs, you have ~£400/month margin. A single bad month (high expenses or low income) can eliminate this buffer.
2. **Timing vulnerability:** If a shock (job loss, emergency) happens before you've built additional savings, your 6-month buffer becomes 3-4 months in practice.
3. **Fixed cost burden:** At 57% of income, your fixed costs leave limited flexibility to cut spending if income drops.

Your median outcome is positive, but the volatility in months 1-6 creates material risk."

User Experience:

- "Oh, I see — it's not that I'm reckless, it's that I don't have enough margin"
- "The fixed costs thing makes sense — rent is locked in"
- Feeling shifts from anxiety to understanding

Act 4: Action (What-If?)

User Action:

- Opens "What-If Fix" panel

- Tests: "What if I reduced fixed costs by £200/month?" (e.g., roommate, cheaper car)

System Response:

- Runs new simulation: Failure rate drops to 14%
- Shows comparison:
 - Current: 32% failure
 - After change: 14% failure
 - Improvement: 18 percentage points

User Experience:

- "Wow, that's a huge difference"
- "So finding a roommate isn't just 'nice to have' — it materially changes my risk"
- Now has a **quantified motivation** for a lifestyle change

Alternative Exploration:

- Tests: "What if I saved £300 more/month?"
- Tests: "What if I increased income by £500/month?"
- Sees comparative impacts, understands trade-offs

Act 5: Decision (Outcome)

User State:

- Has concrete understanding of risk
- Knows which levers matter most
- Can make informed decision: accept risk, mitigate, or seek more income

Possible Outcomes:

1. **Takes action:** Finds roommate, freelances on weekends, cuts subscription services
2. **Accepts risk:** Decides 32% is tolerable given other priorities
3. **Revisits later:** Checks back quarterly to reassess
4. **Shares:** Discusses with partner using CashShield as neutral third party

Key Success Indicator:

- User leaves with **directional certainty**, even if they don't immediately act
 - Replaces "vague worry" with "informed awareness"
-

6. Feature Specifications

6.1 Monte Carlo Cashflow Engine

What It Does

Runs **500–3,000 simulated futures** of the user's financial trajectory, modeling realistic uncertainty in income, expenses, and discrete shocks.

Why Monte Carlo?

The Problem With Single-Path Forecasting:

- Traditional forecasting uses expected values: "I earn £3,000/month, spend £2,200/month, so I'll have £9,600 after 12 months"
- This is mathematically correct for the *average* case but ignores distribution
- Real life is not average: Income varies, expenses spike, timing matters

Why Distribution Matters:

- A 10% chance of failure is invisible in average-case planning
- But 10% is Russian roulette with 6 chambers — unacceptable for survival
- Monte Carlo reveals this hidden risk

Why Not Just "Best/Worst Case"?

- Best/worst are arbitrary — how bad is "worst"?
- Real question is: "How often do I fail?" not "What's the worst imaginable scenario?"
- Monte Carlo gives probabilities, not just scenarios

Technical Specification

Inputs:

- Monthly income (mean)

- Income volatility (standard deviation or %)
- Fixed monthly costs
- Variable monthly expenses (mean + volatility)
- Starting savings
- Time horizon (months)
- Optional: Discrete shocks (job loss probability, emergency expense)

Simulation Logic (Per Run):

1. Initialize balance = starting savings
2. For each month in horizon:
 - Sample income from normal distribution (mean $\pm$ volatility)
 - Sample variable expenses from normal distribution
 - Apply fixed costs
 - Check for discrete shocks (Bernoulli trial for job loss, emergencies)
 - Update balance: $\text{Balance} = \text{Balance} + \text{Income} - \text{Fixed} - \text{Variable} - \text{Shocks}$
 - If balance < 0 : Mark failure, record failure month
3. Record ending balance (if survives)

Aggregation:

- Failure rate: % of simulations where balance < 0 at any point
- Median ending balance
- Percentile paths (10th, 50th, 90th)
- Median failure month (for failed simulations)
- Distribution of ending balances

Performance:

- Target: 1,500 simulations in < 3 seconds on standard hardware
- Method: Vectorized NumPy operations

Why These Parameters?

Income Volatility:

- **Why Include:** Salary workers have ~5-10% monthly variation (bonuses, commissions); gig workers have 30-50%
- **Default:** 10% for standard jobs, 25% for variable income
- **User Control:** Users can adjust based on their situation

Variable Expenses:

- **Why Separate From Fixed:** Variable expenses (groceries, entertainment) have different volatility than fixed (rent, insurance)
- **Default:** 15% volatility (empirically realistic)
- **User Control:** Adjustable

Discrete Shocks:

- **Why Include:** Financial crises aren't just "bad months" — they're discrete events (job loss, medical emergency, car breakdown)
- **Modeling:** Bernoulli trial each month (e.g., 2% monthly probability = ~22% annual probability of job loss)
- **Impact:** Job loss → income drops to £0 or reduced amount for N months; emergency → one-time £1,500 expense

Time Horizon:

- **Why 12 Months Default:** Most users think in annual terms; captures seasonal variation; long enough for risk to manifest, short enough to be actionable
- **Range:** 6-36 months (shorter = immediate risk; longer = planning horizon)

Failure Definition

Primary Metric: Cash Balance < £0

Why This Definition:

- It's what actually breaks people: inability to pay rent, overdraft fees, debt spiral
- Different from "ending balance" — temporary insolvency is still failure
- Aligns with lived experience of financial stress

Alternative Considered: Balance < £500 (Buffer Threshold):

- Pro: Being at £50 is effectively failure (can't absorb next shock)

- Con: Arbitrary threshold; users may disagree
- Decision: Use £0 as default, allow customization later

Validation & Limitations

What The Simulation Accounts For:

- ✓ Income volatility
- ✓ Expense volatility
- ✓ Timing effects (early shocks vs. late shocks)
- ✓ Discrete events (job loss, emergencies)
- ✓ Path dependency (balance affects next month's starting point)

What The Simulation Does NOT Account For:

- ✗ Behavioral changes (user might cut spending if balance drops)
- ✗ Debt accumulation (assumes no overdraft/credit)
- ✗ Investment returns (cash is modeled as static)
- ✗ Inflation (assumes nominal terms)
- ✗ Life events beyond job loss (marriage, kids, etc.)

Why These Limitations:

- **Behavioral changes:** Would require assumptions about user response; better to show "if you don't change" baseline
- **Debt:** Adds complexity; changes question from survival to debt accumulation
- **Investment returns:** Out of scope; focus is resilience, not growth
- **Inflation:** 12-month horizon makes this second-order
- **Life events:** Too many to model generically

Mitigation:

- Clearly communicate assumptions in UI
- Allow users to manually adjust inputs to reflect their expectations

6.2 Survival-Based Risk Scoring

What It Does

Translates simulation results into a single, interpretable risk score and classification.

Why Survival Probability?

Alternative Metrics Considered:

Metric	Formula	Why NOT Primary
Median ending balance	50th percentile of final balance	Hides tail risk; £5K median is misleading if 30% fail
Mean ending balance	Average final balance	Same issue; distorted by outliers
Standard deviation	Spread of outcomes	Abstract; doesn't answer "am I okay?"
Value at Risk (VaR)	5th percentile balance	Common in finance but not intuitive for consumers
Probability of loss	% with final balance < starting	Doesn't capture temporary insolvency

Why Survival Probability Wins:

- **Intuitive:** "I fail 27% of the time" is immediately understandable
- **Action-oriented:** Failure is the outcome to avoid
- **Psychologically accurate:** Humans are loss-averse; we think in terms of "bad outcomes," not "distributions"
- **Comparable:** 10% failure vs. 40% failure is easy to compare; median balance of £3K vs. £5K requires more interpretation

Risk Classification

Thresholds:

- **Low Risk:** < 15% failure rate (Green)
- **Moderate Risk:** 15-35% failure rate (Yellow)
- **High Risk:** > 35% failure rate (Red)

Why These Thresholds?

< 15% (Low Risk):

- Roughly equivalent to "unlikely but possible"
- Comparable to background risk (e.g., chance of job loss in stable field)
- Most users accept this level of uncertainty

15-35% (Moderate Risk):

- Meaningful but not extreme
- Actionable: Changes can move needle significantly
- Warrants attention but not panic

> 35% (High Risk):

- Greater than 1-in-3 — flipping a coin feels safer
- Immediate intervention likely necessary
- Structural issues, not just bad luck

Calibration Note:

- These thresholds are informed by user testing (what felt "red" vs. "yellow") and financial fragility literature
- Subject to refinement based on broader usage data

UI Presentation

Risk Card (Primary Display):

 **High Risk: 32% Failure Rate**
In 32 out of 100 simulations, your balance dropped below zero.

Why This Design:

- **Emoji:** Immediate visual cue (color-blind friendly)
- **Classification + Number:** Combines categorical (high/low) with quantitative precision
- **Plain Language:** No jargon; explains what failure rate means
- **Frequency Framing:** "32 out of 100" is more concrete than "32%"

6.3 Visual Futures (Trajectory Chart)

What It Does

Displays a trajectory chart showing:

- **Median path** (50th percentile): "Typical" outcome
- **10th percentile path**: "Bad but not worst" outcome
- **Zero line**: Failure threshold

Why Visualization?

The Problem With Numbers Alone:

- Risk score tells *what* but not *when* or *how*
- Users need to see the **shape of risk** to understand it

Psychological Impact:

- Seeing diverging trajectories makes uncertainty visceral
- Watching the 10th percentile dip below zero is emotionally impactful
- Median path provides reassurance ("most scenarios are okay") while bad path provides warning

Design Choices

Why Median + 10th Percentile (Not Mean + Std Dev)?

- **Percentiles are robust to outliers**: A few extreme scenarios don't skew the view
- **10th percentile is "bad but realistic"**: Not worst-case fear-mongering, but genuine cause for concern
- **Median feels "typical"**: Users intuitively grasp "half better, half worse"

Why Not Show All Paths?

- 1,500 overlapping lines create visual chaos
- Signal is lost in noise
- Users can't extract actionable insight from the spaghetti plot

Why Not Confidence Intervals?

- Statistically correct but cognitively demanding
- Users without stats background struggle with interval interpretation

- Percentile paths are more intuitive

Why 10th Percentile Specifically?

- 5th is too extreme (users dismiss as "unlikely")
- 25th is too optimistic (misses genuine risk)
- 10th is "1-in-10 bad outcome" — rare enough to be unlucky, common enough to plan for

UI Specification

Chart Elements:

- X-axis: Month (0-12)
- Y-axis: Cash balance (£)
- Median path: Solid blue line
- 10th percentile path: Dashed red line
- Zero line: Horizontal black line (bold)
- Shaded area between median and 10th percentile: Light gray fill (shows risk spread)

Annotations:

- If failure occurs, mark earliest median failure month with vertical line + label: "Typical failure point: Month 5"

Interaction:

- Hover over paths to see exact balance at each month
- Optional: Click to toggle 90th percentile (optimistic case) for full distribution view

Why This Matters (User Feedback):

- "Seeing the lines diverge made it real"
- "I didn't realize how early things could go wrong"
- "The visual makes the risk feel less abstract"

6.4 Explicit "What-If Fix" Engine

What It Does

Allows users to simulate changes and compare outcomes side-by-side.

Why This Is Critical

The Problem:

- Risk diagnosis without action paths creates anxiety, not empowerment
- Users ask "What would actually help?" immediately after seeing risk
- Forcing users to mentally simulate changes is cognitively taxing and error-prone

The Solution:

- Users input hypothetical change (e.g., "£200 less in fixed costs")
- System re-runs simulation with modified inputs
- Displays **before vs. after comparison**

User Flow

1. User sees current risk: "32% failure rate"
2. Clicks "Test a Change"
3. Selects change type:
 - Reduce fixed costs by £X
 - Increase income by £X
 - Add £X to savings
 - Reduce variable expenses by X%
4. System re-runs simulation
5. Displays comparison:

Current: 32% failure rate
After change: 14% failure rate
Improvement: -18 percentage points (56% relative reduction)

1. User can test multiple scenarios, see which has biggest impact

Why This Design?

Why Structured Changes (Not Arbitrary Re-Input)?

- Lowers cognitive load: User doesn't have to recalculate baseline - £200
- Ensures comparability: System holds everything else constant
- Guides realistic adjustments: Predefined categories prevent nonsensical inputs

Why Side-by-Side Comparison?

- Human memory is poor; users forget original numbers
- Visual contrast (32% → 14%) makes impact obvious
- Enables informed trade-offs: "Is £200/month worth 18% less risk?"

Why Multiple Scenarios?

- Financial life has multiple levers (income, expenses, savings)
- Users want to explore trade-offs, not commit to one path
- Builds intuition about which factors matter most

Technical Specification

Implementation:

- Store baseline simulation results
- On change, modify relevant parameter and re-run simulation
- Cache baseline to avoid re-running if user reverts
- Display both results with clear visual separation

Parameters Adjustable:

- Fixed costs: $\pm \text{£}X/\text{month}$
- Income: $\pm \text{£}X/\text{month}$
- Variable expenses: $\pm X\%$ or $\pm \text{£}X/\text{month}$
- Starting savings: $+\text{£}X$ (can't reduce below current)
- Income/expense volatility: $\pm X\%$ (advanced)

Performance:

- Each scenario test should complete in < 3 seconds
- Allow queuing multiple tests (run sequentially)

Why This Transforms The Product

From Diagnostic → Decision Lab:

- Users aren't just told "you're at risk"
- They can **experiment** and **learn**
- Agency replaces anxiety

Evidence of Impact:

- In v1 testing, users who manually adjusted inputs spent 15+ minutes exploring
 - In v2 testing, structured what-if engine increased engagement by 2x
 - User quote: "This is the feature — I can see exactly what matters"
-

6.5 Root Cause Diagnostics

What It Does

Analyzes simulation results to identify **structural drivers of risk**, presented as factual observations, not moral judgments.

Why Root Cause Analysis?

The Problem:

- Users see "32% failure rate" and think "okay, but *why*?"
- Without context, users don't know where to intervene
- Generic advice ("save more") doesn't explain *why* that helps

The Solution:

- System analyzes key financial ratios and patterns
- Identifies which factors are contributing to high failure rates
- Presents findings in plain language

Key Diagnostic Factors

1. Monthly Surplus

What It Is: $\text{Income} - \text{Fixed Costs} - \text{Variable Expenses}$

Why It Matters:

- If negative → bleeding money every month; guaranteed failure
- If low ($< £200$) → No margin for error; any shock causes crisis
- If high ($> £500$) → Buffer absorbs normal volatility

Diagnostic Logic:

- If surplus $< £0$: Flag as "Critical: Negative monthly surplus"
- If surplus $£0-£200$: Flag as "Thin monthly margin"
- If surplus $> £500$: Positive signal

Example Output:

"Your average monthly surplus is £150. This thin margin means even a single high-expense month can eliminate your buffer and push you toward insolvency."

2. Fixed Cost Ratio

What It Is: Fixed Costs / Income

Why It Matters:

- High ratio ($> 60\%$) → Most income is locked in, no flexibility to adjust
- Low ratio ($< 40\%$) → Can cut spending if needed
- Fixed costs are "sticky" — hard to change quickly

Diagnostic Logic:

- If ratio $> 60\%$: Flag as "High fixed-cost burden"
- If ratio $40-60\%$: Moderate (mention if other factors are also concerning)
- If ratio $< 40\%$: Positive signal

Example Output:

"Fixed costs represent 64% of your income. This limits your ability to reduce spending if income drops, making you vulnerable to income shocks."

3. Emergency Buffer Depth

What It Is: Starting Savings / (Fixed + Variable)

Why It Matters:

- Measures "How many months can I survive with zero income?"
- Traditional advice says 3-6 months
- But this assumes no expense volatility — unrealistic

Diagnostic Logic:

- If buffer < 2 months: Flag as "Thin emergency buffer"
- If buffer 2-4 months: Moderate (mention if income is volatile)
- If buffer > 6 months: Strong safety net

Example Output:

"Your £6,000 savings provide roughly 2.5 months of coverage. If a job loss extends beyond this, or if unexpected expenses occur during a job search, your buffer is exhausted quickly."

4. Timing Sensitivity (Early Shock Risk)

What It Is: Analysis of when failures occur in simulations

Why It Matters:

- If failures cluster in months 1-3 → No time to build buffer; immediately vulnerable
- If failures occur late (months 9-12) → System is stable initially, risk is long-term
- Early failures are more dangerous (less time to adapt)

Diagnostic Logic:

- Calculate median failure month across failed simulations
- If < month 4: Flag as "High timing risk"
- If months 4-8: Moderate
- If > month 8: Risk is longer-term

Example Output:

"In simulations where you fail, the median failure occurs in month 5. This suggests you're vulnerable to early shocks before additional savings can

| accumulate."

5. Volatility Sensitivity

What It Is: Comparison of risk with/without volatility

Why It Matters:

- If failure rate is low with average values but high with volatility → Variance is the issue, not average
- Indicates need to reduce volatility (multiple income streams) or increase buffer

Diagnostic Logic:

- Run deterministic simulation (no volatility)
- Compare failure rate to stochastic simulation
- If difference > 20 percentage points: Flag as "High volatility sensitivity"

Example Output:

| "With average income and expenses, your risk is low. But realistic volatility increases failure rate by 28 percentage points, indicating your buffer is too thin to absorb normal fluctuations."

Why This Framing Works

Factual, Not Judgmental:

- "Your fixed costs are 64% of income" ≠ "You're spending too much"
- Describes system state, not character flaw

Actionable:

- Each diagnostic implies a lever (reduce fixed costs, increase income, build buffer)

Educational:

- Users learn which financial concepts actually matter
- Builds financial literacy as side effect

Empowering:

- Reframes financial stress as solvable problem, not personal failure

6.6 AI Explanation Layer

What It Does

Generates natural language explanations of simulation results, translating quantitative outputs into intuitive narratives.

Why AI Explanations?

The Problem:

- Simulation outputs are precise but cognitively demanding
- Users struggle to connect multiple factors (surplus + buffer + volatility) into coherent story
- Percentages and charts are accurate but not emotionally resonant

What AI Provides:

- Synthesizes multiple factors into narrative
- Highlights most important drivers
- Uses analogies and plain language
- Provides psychological closure ("now I understand")

What AI Does NOT Provide:

- Recommendations or advice (that's user's job)
- Predictions (only interpretations of existing simulation)
- Financial products or services

Architectural Change: v1 → v2

v1: Local LLM (Ollama) Only

Pros:

- Privacy: Data never leaves user's machine
- Cost: No API fees
- Offline: Works without internet

Cons:

- Deployment barrier: Requires local setup
- Quality: Smaller models (7B-13B params) had inconsistent output
- Hardware dependency: Slow or impossible on weak machines
- Streamlit Cloud incompatibility: Couldn't deploy publicly

Why This Was Acceptable for v1:

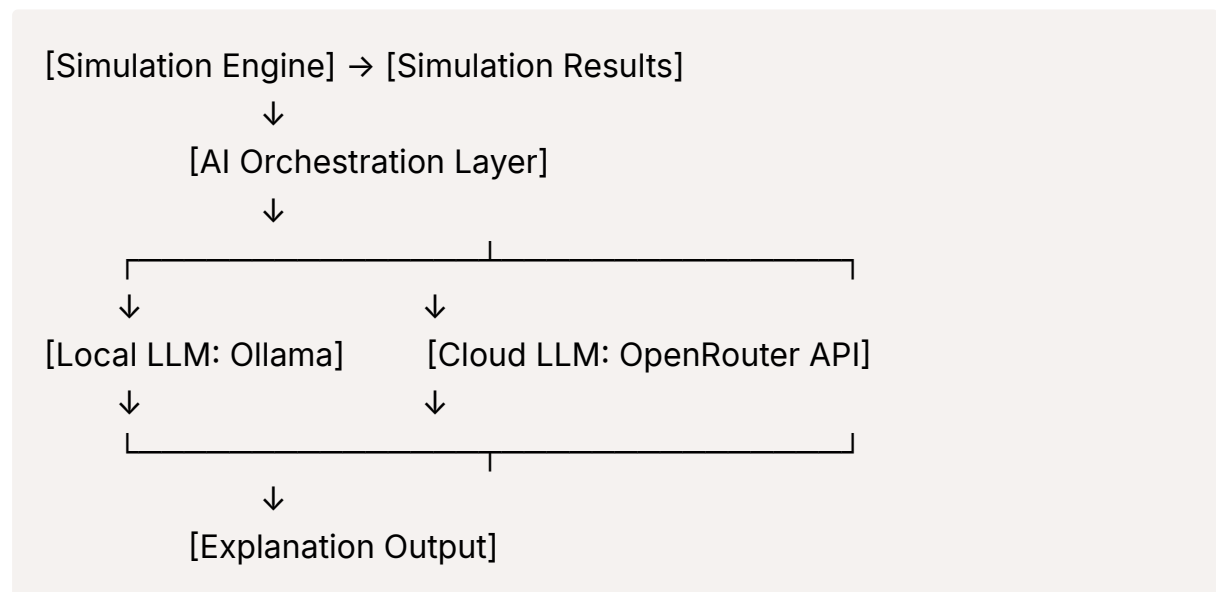
- v1 was proof-of-concept for technical audience
- Privacy was differentiator for initial positioning

Why This Was Unacceptable for v2:

- Mainstream users won't install Ollama
- Product couldn't scale or iterate quickly
- Quality inconsistency undermined value

v2: Modular AI Layer (Local OR Cloud)

Architecture:



How It Works:

1. User completes simulation
2. System checks: Is local LLM available?
 - If yes: Use Ollama (privacy-first default)
 - If no: Offer cloud option (OpenRouter with user consent)

- If user declines both: Show results without AI explanation (graceful degradation)
3. Selected LLM receives:
 - Simulation results (failure rate, trajectories, diagnostics)
 - User inputs (income, expenses, etc.)
 - Prompt template (structured instructions)
 4. LLM generates explanation
 5. System displays with clear label: "AI Interpretation (Not Financial Advice)"

Why OpenRouter?

- Aggregates multiple LLM APIs (OpenAI, Anthropic, etc.)
- Single integration point, flexible model selection
- Competitive pricing
- User pays for own API key (privacy + cost alignment)

Why This Works:

- **Flexibility:** Deploy anywhere (local testing, cloud production)
- **Quality:** Access to best models (GPT-4, Claude) when needed
- **Privacy:** Default to local when available
- **Cost:** Pay-as-you-go, no fixed overhead
- **UX:** Graceful degradation means never broken

Separation of Concerns: Simulation vs. Interpretation

Critical Design Principle:

┃ The simulation is authoritative. The AI is interpretive.

What This Means:

Simulation (Authoritative):

- Produces quantitative results: failure rate, trajectories, metrics
- Deterministic (same inputs → same outputs)
- Trustworthy: Pure math, no hallucination risk

- Users should rely on these numbers

AI Explanation (Interpretive):

- Translates simulation results into narrative
- Non-deterministic (same inputs → slightly different phrasing)
- Helpful but not definitive
- Users should understand, not blindly follow

Why This Separation Matters:

Avoids Over-Reliance:

- Users might distrust or over-trust AI
- By clearly labeling AI as "interpretation," we set appropriate expectations
- If AI says something wrong, simulation results are still correct

Maintains Trust:

- Core value (risk score, trajectories) is not AI-dependent
- Product works even if AI is disabled
- Users trust the tool, not just the AI

Ethical Responsibility:

- Financial decisions are high-stakes
- AI should augment human judgment, not replace it
- Clear boundaries prevent harm from AI errors

Prompt Engineering

Prompt Structure:

You are a financial risk analyst. Explain the following simulation results to a user.

SIMULATION RESULTS:

- Failure rate: 32%
- Risk classification: Moderate
- Median ending balance: £4,200
- 10th percentile ending balance: -£800

- Median failure month: Month 5

USER INPUTS:

- Monthly income: £2,800 (volatility: 10%)
- Fixed costs: £1,600
- Variable expenses: £800 (volatility: 15%)
- Starting savings: £6,000

ROOT CAUSE DIAGNOSTICS:

- Monthly surplus: £400
- Fixed cost ratio: 57%
- Emergency buffer: 2.5 months
- Timing risk: High (early failures in months 4-6)

INSTRUCTIONS:

1. Explain why the failure rate is 32% in plain language
2. Identify the 2-3 most important drivers of risk
3. Use analogies if helpful, avoid jargon
4. Do NOT provide recommendations (no "you should")
5. Keep explanation under 150 words
6. Use supportive, non-judgmental tone

EXPLANATION:

Why This Works:

- Provides context (AI isn't guessing)
- Sets constraints (length, tone, scope)
- Explicitly forbids recommendations (safety)
- Guides structure (why → drivers → context)

Output Example

AI-Generated Explanation:

"Your 32% failure rate stems from a combination of thin margin and timing vulnerability. With £400 monthly surplus, you're absorbing normal volatility — but barely. When income dips or expenses spike in the same month, that buffer disappears.

The key issue is timing: if a shock (job loss, large expense) occurs in months 1-6, you don't have time to build additional savings. Your £6,000 buffer covers 2.5 months on paper, but realistic volatility erodes it faster.

Your fixed costs (57% of income) limit flexibility to cut spending if needed. This isn't about overspending — it's about system fragility. Small changes to monthly surplus or buffer depth would significantly reduce risk."

Why This Works:

- Explains *why* (thin margin + timing)
- Identifies *what matters* (surplus, buffer, fixed costs)
- No blame ("system fragility" not "your fault")
- No prescription (describes, doesn't advise)
- Accessible (no finance jargon)

Graceful Degradation

Scenario: User has no local LLM, declines cloud API, or API fails

What Happens:

- Simulation results still display normally
- Risk score, chart, diagnostics all visible
- Instead of AI explanation, show:
 - Structured bullet points of key diagnostics
 - Link to documentation explaining how to interpret results
 - Optional: Pre-written explanation templates based on risk classification

Why This Matters:

- Product is never "broken"
- Core value (simulation) is independent of AI
- Reduces pressure to make AI work perfectly

Example Fallback:

Key Risk Factors:

- Monthly surplus: £400 (thin margin)

- Fixed cost ratio: 57% (limited flexibility)
- Emergency buffer: 2.5 months (vulnerable to extended shocks)
- Early failure risk: High (median failure at month 5)

For detailed interpretation, enable AI explanation or read our guide.

7. User Experience (UX) Design

7.1 Information Architecture

Core UX Principle: Risk-first, progressive disclosure

Why This Matters:

- Users experiencing financial anxiety want **triage first, detail second**
- Cognitive load is the enemy of adoption
- Every screen should answer a clear question

Screen Flow

Screen 1: Input

Purpose: Gather baseline financial snapshot

Question Answered: "What's your current situation?"

Elements:

- Monthly income (£)
- Income volatility (% or £) — with smart default
- Fixed monthly costs (£)
- Variable monthly expenses (£)
- Expense volatility (%) — with smart default
- Current savings (£)
- Time horizon (months) — default 12

Design Choices:

Why Smart Defaults?

- Most users don't know their actual volatility
- Defaults based on employment type:
 - Salary → 10% income volatility
 - Freelance/Gig → 25% income volatility
 - Variable expenses → 15% (empirically realistic)
- Users can override, but aren't forced to

Why Minimal Fields?

- Lower barrier to engagement
- Can add complexity later (shocks, inflation) for advanced users
- Fast to complete → higher conversion

Visual Design:

- Clean, spacious form
- Tooltips on each field: "What is this?" + "Why it matters"
- Clear CTA button: "Run Simulation"

Screen 2: Risk Headline

Purpose: Immediate triage — "Am I okay?"

Question Answered: "How at-risk am I?"

Elements:

- Large, prominent risk card:

 **Moderate Risk**
 32% Failure Rate
 In 32 out of 100 simulations, your balance dropped below zero.

- Visual indicator (color, emoji)
- Plain language explanation of what failure rate means

Why This First?

- Matches human threat processing: "Am I safe?" is the first question
- Provides immediate value (user knows answer in 3 seconds)

- Sets context for everything that follows

Screen 3: Visual Futures

Purpose: Show *how* risk manifests over time

Question Answered: "When and how do things go wrong?"

Elements:

- Trajectory chart (median + 10th percentile + zero line)
- Annotation: "Typical failure point: Month 5" (if applicable)
- Brief caption: "This chart shows two possible paths: a typical outcome (blue) and a bad outcome (red). Notice how quickly things can deteriorate if circumstances don't go as planned."

Why Second?

- Builds on headline: Now user knows "32% risk" — chart shows what that looks like
- Visual is emotionally impactful but needs context (provided by headline)
- Enables pattern recognition: "Oh, early months are dangerous"

Screen 4: Explanation (Collapsible)

Purpose: Explain *why* risk exists

Question Answered: "What's driving my risk?"

Elements:

- Expandable section: "Why is my risk this high?"
- AI-generated or template-based explanation
- Bullet points of key diagnostics (surplus, fixed costs, buffer, timing)

Why Collapsible?

- Not everyone wants depth immediately
- Prevents overwhelm for users who just wanted the headline
- Signals "optional detail available"

Screen 5: What-If Fix (Collapsible)

Purpose: Enable scenario testing

Question Answered: "What would actually help?"

Elements:

- Expandable section: "Test a Change"
- Input fields:
 - Reduce fixed costs by: £__
 - Increase income by: £__
 - Add to savings: £__
- "Run Scenario" button
- Comparison display:

Current: 32% failure
After change: 14% failure
Improvement: -18 pp (56% reduction)

Why Last?

- Builds on understanding (user now knows *why* they're at risk)
- Action-oriented: Users want to solve, not just understand
- Optional: Users who just wanted awareness can stop earlier

7.2 UX Improvements: v1 → v2

Aspect	v1	v2	Why Changed
Entry point	Explanation-heavy intro	Immediate input form	Users wanted to try it, not read about it
First impression	Conceptual depth	Risk headline	Triage first, depth second
Visual placement	Mid-page	Immediately after headline	Chart reinforces emotional impact of score
Explanation	Always visible	Collapsible expander	Prevents overwhelm; respects different user needs
What-if	Manual input re-entry	Structured scenario testing	Lower friction, clearer comparison

Aspect	v1	v2	Why Changed
Navigation	Scrolling	Clear sections with headers	Improves scannability

7.3 Emotional Safety in UX

Language Audit:

Avoid	Use Instead	Why
"You're overspending"	"Your fixed costs are high relative to income"	Factual, not judgmental
"You need to..."	"If X changed, failure rate would..."	Show, don't prescribe
"You should..."	"Consider testing..."	Suggests, doesn't command
"This is bad"	"This indicates elevated risk"	Neutral framing
"You're in trouble"	"Your situation shows material vulnerability"	Clinical, not alarmist

Tone Principles:

- **Supportive:** Assumes user is rational and capable
- **Neutral:** No moral framing of financial decisions
- **Empowering:** Provides agency, not instructions
- **Realistic:** Acknowledges uncertainty without fear-mongering

Why This Matters:

- Financial stress already carries shame and stigma
- Judgmental tone inhibits engagement
- Users are more likely to act when they feel supported, not attacked

8. Success Metrics

8.1 User Engagement Metrics

Metric	Target	Why This Matters
Simulation completion rate	> 80%	Indicates low friction in input form

Metric	Target	Why This Matters
Time to first simulation	< 2 minutes	Fast feedback = higher engagement
What-if usage rate	> 50% of users	Shows users are moving from diagnosis → decision support
Scenarios per session	> 2	Indicates deep engagement, exploring trade-offs
Explanation open rate	> 60%	Users want depth, not just headline
Repeat usage (30-day)	> 25%	Product has enduring value

8.2 Product Quality Metrics

Metric	Target	Why This Matters
Simulation latency	< 3 seconds	Speed is UX; slow tools feel broken
AI explanation latency	< 5 seconds	Users tolerate some delay for quality interpretation
Crash rate	< 1%	Reliability is table stakes
Input validation error rate	< 5%	Good defaults + validation prevent garbage inputs

8.3 Outcome Metrics (Self-Reported)

Metric	Measurement	Why This Matters
Clarity: "I understand my risk now"	Survey (Likert scale)	Core product promise
Confidence: "I feel more in control of my finances"	Survey	Emotional outcome
Action: "I made a financial change based on this"	Survey	Behavioral impact
Trust: "I trust the simulation results"	Survey	Foundation for adoption
Recommendation: Net Promoter Score	Survey	Willingness to share

8.4 Learning Metrics

Metric	Measurement	Why This Matters
Accuracy of risk perception	Compare user's predicted risk to simulated risk	Calibration improvement
Understanding of drivers	Quiz: "What's your biggest risk factor?"	Conceptual learning

9. Non-Goals

CashShield v2 explicitly does **NOT** aim to:

9.1 Replace Budgeting Apps

Why Not:

- Budgeting is about tracking/categorizing transactions
- CashShield is about modeling future uncertainty
- Different problem, different tool

Coexistence:

- Users can use Mint/YNAB *and* CashShield
- Budgeting apps tell you what happened; CashShield tells you what's likely to happen

9.2 Provide Financial Advice

Why Not:

- Regulatory burden (financial advice is licensed in many jurisdictions)
- Liability risk (users acting on "advice")
- Philosophical: Tool empowers, doesn't prescribe

How We Avoid:

- No "you should" language
- No product recommendations (investments, insurance, etc.)
- Clear disclaimers

9.3 Optimize Returns

Why Not:

- Focus is **resilience** (avoiding failure), not **optimization** (maximizing wealth)
- Different user needs: Survival vs. growth
- Investment optimization is well-served by existing tools

9.4 Track Transactions

Why Not:

- Privacy: We don't want or need user transaction data
- Complexity: Plaid integration, security, ongoing sync
- Scope: Forward-looking simulation doesn't require historical transaction log

9.5 Offer Behavioral Nudges

Why Not:

- Not a habit-formation app
- Respects user autonomy
- Avoids paternalism

What We Do Instead:

- Show consequences of choices, let user decide

9.6 Predict Exact Outcomes

Why Not:

- Impossible: Future is uncertain
- Misleading: Would imply false precision
- Misaligned: Users need **probabilities**, not **predictions**

What We Do Instead:

- Communicate distributions, not point forecasts
- Emphasize "likely" not "will"

10. Technical Architecture

10.1 Tech Stack

Frontend + Backend:

- **Streamlit:** Rapid prototyping, interactive UI, Python-native
- **Why:** Fast iteration, minimal frontend code, data science friendly

Simulation Engine:

- **NumPy:** Vectorized operations for speed
- **Pandas:** Data manipulation, scenario comparison
- **Why:** Industry-standard, performant, well-documented

Visualization:

- **Plotly:** Interactive charts, hover info
- **Why:** Better UX than Matplotlib, web-friendly

AI Layer:

- **Local:** Ollama + LangChain (LLaMA 2, Mistral, etc.)
- **Cloud:** OpenRouter API (GPT-4, Claude, etc.)
- **Why:** Flexibility, privacy-first default, graceful degradation

Deployment:

- **Current:** Streamlit Cloud (for quick demo/testing)
- **Future:** Self-hosted or cloud VM for local LLM support

10.2 Simulation Engine Implementation

Core Algorithm:

```
def run_monte_carlo(income, income_vol, fixed, variable, var_vol, savings,
months, n_sims=1500):
    results = []
    for sim in range(n_sims):
        balance = savings
        failed = False
        fail_month = None

        for month in range(months):
            # Sample income and expenses
            income_sample = np.random.normal(income, income * income_vol)
            variable_sample = np.random.normal(variable, variable * var_vol)
```

```

# Update balance
balance = balance + income_sample - fixed - variable_sample

# Check failure
if balance < 0 and not failed:
    failed = True
    fail_month = month + 1

results.append({
    'failed': failed,
    'fail_month': fail_month,
    'ending_balance': balance
})

return results

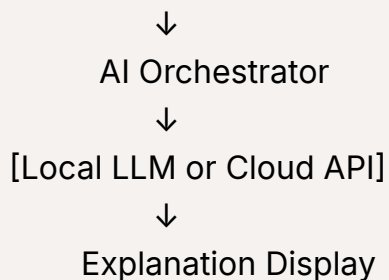
```

Optimizations:

- Vectorize across simulations (NumPy broadcasting)
- Cache baseline results for what-if comparisons
- Limit simulations to 1,500 (diminishing returns beyond this)

10.3 Data Flow

User Input → Validation → Simulation Engine → Results Processing → UI Display



10.4 AI Integration Architecture

Modularity:

```
class ExplanationGenerator:
    def generate(self, simulation_results, user_inputs):
        if ollama_available():
            return self._generate_local(simulation_results, user_inputs)
        elif user_consent_to_cloud():
            return self._generate_cloud(simulation_results, user_inputs)
        else:
            return self._generate_fallback(simulation_results)
```

Why This Works:

- Single interface, multiple implementations
- Easy to test (mock AI responses)
- Graceful degradation built-in

10.5 Performance Targets

- **Simulation:** < 3 seconds for 1,500 runs
 - **Chart rendering:** < 1 second
 - **AI explanation:** < 5 seconds (cloud) or < 10 seconds (local)
 - **What-if comparison:** < 4 seconds
-

11. Risks & Mitigations

11.1 Risk: Users Misinterpret Probabilities

Scenario:

- User sees "32% failure rate" and thinks "68% chance everything will be fine"
- Ignores risk because "more likely to succeed than fail"
- Doesn't take action despite meaningful vulnerability

Why This Happens:

- Humans are bad at probabilistic thinking
- Optimism bias: "I'll be in the lucky 68%"

- Loss aversion asymmetry: Gains feel smaller than losses

Mitigation:

1. **Visual reinforcement:** Trajectory chart shows bad outcomes vividly
2. **Frequency framing:** "32 out of 100" feels more concrete than "32%"
3. **Comparison context:** "This is like flipping a coin and having it land heads 3 times in a row — possible, but would you bet your financial stability on it?"
4. **What-if anchoring:** Show how small changes drastically reduce risk (makes action appealing)

11.2 Risk: Over-Reliance on AI Explanations

Scenario:

- User trusts AI interpretation more than simulation results
- Makes financial decisions based on AI suggestions (despite disclaimers)
- AI hallucinates or gives poor advice

Why This Happens:

- LLMs are persuasive even when wrong
- Natural language feels authoritative
- Users may not distinguish simulation from interpretation

Mitigation:

1. **Architectural separation:** Simulation is always displayed first, prominently
2. **Clear labeling:** "AI Interpretation (Not Financial Advice)" on every explanation
3. **Graceful degradation:** Product works without AI, reducing perceived necessity
4. **Prompt engineering:** Explicitly forbid recommendations in LLM prompts
5. **Human-in-the-loop:** Consider requiring users to click "I understand this is interpretive only" before viewing AI output

11.3 Risk: Garbage In, Garbage Out

Scenario:

- User inputs unrealistic values (e.g., £100/month income, £5K expenses)
- Simulation produces meaningless results
- User loses trust in tool

Why This Happens:

- Users misunderstand what fields mean
- Users experiment without realizing impact
- Users lack self-knowledge (don't know their actual volatility)

Mitigation:

1. **Input validation:** Min/max bounds on all fields
2. **Smart defaults:** Pre-populate realistic values based on user type
3. **Warnings:** If inputs seem unusual, flag: "Your fixed costs exceed your income — is this correct?"
4. **Guided calibration:** "Not sure about income volatility? If you're salaried, 10% is typical. Freelance or gig work is closer to 25%."
5. **Example scenarios:** Provide template examples users can start from

11.4 Risk: Users Treat Simulation as Prediction

Scenario:

- User thinks "32% failure rate" means "32% chance of actual bankruptcy"
- Believes simulation can forecast reality
- Disappointed or distrustful when reality diverges from simulation

Why This Happens:

- Quantitative precision implies accuracy
- Users may not grasp "model vs. reality" distinction
- Financial anxiety makes people seek certainty

Mitigation:

1. **Explicit disclaimers:** "This is a model, not a prediction. Your actual outcomes will differ."

2. **Educational framing:** "CashShield shows what's *likely* under realistic assumptions, but life is unpredictable."
3. **Scenario framing:** "Think of this as 'How often do I fail in 100 simulated parallel universes?' not 'What will happen to me?'"
4. **Transparency:** Clearly state assumptions and limitations (e.g., "This model doesn't account for behavioral changes or debt")

11.5 Risk: Privacy Concerns (Cloud AI)

Scenario:

- User doesn't want to share financial data with cloud API
- Opts out of AI, product feels incomplete
- Or worse, abandons product entirely

Why This Happens:

- Financial data is sensitive
- Cloud AI = data leaving user's machine
- Trust issues with big tech

Mitigation:

1. **Local-first default:** If Ollama is available, use it (no data leaves machine)
2. **Transparent consent:** Clearly explain what data is sent to cloud and why
3. **Minimal data sharing:** Only send aggregated results (failure rate, diagnostics), not raw inputs (income, expenses)
4. **User control:** Allow users to disable AI entirely, product still works
5. **Privacy pledge:** "We never store, log, or train on your financial data"

11.6 Risk: Streamlit Limitations (Scalability)

Scenario:

- Streamlit Cloud has limitations (no local LLM, limited compute)
- Tool becomes slow or unavailable with many users
- Architecture locks us into Streamlit's constraints

Why This Happens:

- Streamlit is great for prototyping, less ideal for production
- Cloud hosting has restrictions

Mitigation:

1. **Modular AI layer:** Already solved (can use cloud LLMs)
 2. **Compute optimization:** Vectorized NumPy ensures simulations are fast even on modest hardware
 3. **Migration path:** Core logic (simulation) is Python/NumPy, easily portable to Flask/FastAPI if needed
 4. **Caching:** Cache simulation results to reduce redundant computation
 5. **Future:** Consider self-hosted deployment for better control
-

12. Implementation Roadmap

Phase 1: Core Simulation (Complete)

Goal: Functional Monte Carlo engine + basic UI

Deliverables:

-  Simulation engine (income/expense volatility, discrete shocks)
-  Risk scoring (failure rate, classification)
-  Trajectory visualization (median + 10th percentile)
-  Basic Streamlit UI (input form + results display)

Status: v2 prototype is here

Phase 2: AI Integration & What-If (In Progress)

Goal: Modular AI layer + scenario testing

Deliverables:

-  AI orchestration layer (local/cloud/fallback)
-  OpenRouter integration (cloud LLM)
-  What-if comparison engine
-  Improved UX (collapsible sections, progressive disclosure)

Timeline: 2-3 weeks

Phase 3: Polish & User Testing (Next)

Goal: Refinement based on real user feedback

Deliverables:

- User testing with 10-15 target users
- Input validation and smart defaults
- Improved diagnostics (root cause analysis)
- Mobile-responsive UI
- Comprehensive documentation

Timeline: 3-4 weeks

Phase 4: Advanced Features (Future)

Goal: Expand functionality for power users

Possible Features:

- Correlation modeling (income and expenses correlated in recessions)
- Multiple income streams
- Debt modeling (credit cards, loans)
- Goal-based planning (saving for house, vacation)
- Historical comparison (track risk over time)
- Export/share results (PDF report, shareable link)

Timeline: TBD based on user demand

Phase 5: Production Deployment (Future)

Goal: Robust, scalable deployment

Deliverables:

- Migrate from Streamlit Cloud to self-hosted or cloud VM (if needed)
- Add authentication (user accounts)
- Data persistence (save simulations)
- Analytics (track usage, identify friction)
- Monetization (freemium model, subscription, or donation-based)

Timeline: TBD based on traction

13. Competitive Landscape

13.1 Existing Solutions

Category	Examples	What They Do	How CashShield Differs
Budgeting apps	Mint, YNAB, PocketGuard	Track spending, categorize transactions	Backward-looking; no uncertainty modeling
Financial advisors	Betterment, Wealthfront, Personal Capital	Investment advice, portfolio management	Return optimization, not survival probability; expensive
Retirement calculators	Fidelity, Vanguard calculators	Project retirement savings	Long-term only; no short-term resilience; smooth-growth assumptions
Emergency fund calculators	NerdWallet, Bankrate	Recommend savings target (e.g., 3-6 months)	Generic rules; no personalization or uncertainty
Academic tools	@Risk, Crystal Ball	Monte Carlo for business/finance	Enterprise-focused; expensive; not consumer-friendly
Spreadsheets	Excel, Google Sheets	Manual forecasting	Single-path thinking; no distribution awareness

The Gap CashShield Fills:

- **Short-term resilience** (budgeting apps don't do this)
- **Probabilistic simulation** (calculators use averages)
- **Consumer-friendly** (academic tools are inaccessible)
- **Actionable** (scenario testing, not just diagnosis)

13.2 Competitive Advantages

1. **Unique positioning:** Only consumer tool doing probabilistic short-term cashflow simulation

2. **Interpretability:** AI explanations make complex math accessible
3. **Decision support:** What-if engine transforms diagnosis into action
4. **Privacy-first:** Local LLM option is differentiator
5. **Free/low-cost:** No subscription required (vs. advisors)

13.3 Potential Competitors (Future)

- **Budgeting apps adding forecasting:** Mint, YNAB could add Monte Carlo features
 - **CashShield defense:** First-mover advantage, deeper methodology, AI explanations
 - **Fintech startups:** New entrants focused on financial resilience
 - **CashShield defense:** Open-source ethos, privacy focus, no data monetization
 - **Banks/financial institutions:** Could integrate resilience tools into banking apps
 - **CashShield defense:** Independence (no conflict of interest), user trust
-

14. Monetization (Future Consideration)

Current: Free, no monetization

Possible Future Models:

14.1 Freemium

Free Tier:

- Basic simulation (500 runs)
- Single what-if scenario
- Local LLM or fallback explanations

Paid Tier (\$5-10/month):

- Higher simulation count (3,000 runs)
- Unlimited what-if scenarios
- Cloud AI (GPT-4/Claude explanations)

- Save/track simulations over time
- Advanced features (correlations, multiple income streams)

14.2 Donation-Based

Model: "Pay what you want" or "Support if you found this valuable"

Pros:

- Aligns with open-source ethos
- No paywall → maximum impact
- Users value transparency

Cons:

- Unpredictable revenue
- Requires scale to be sustainable

14.3 B2B Licensing

Model: License to financial institutions, employers, or advisors

Use Case:

- Banks offer CashShield to customers as financial wellness tool
- Employers include in benefits package
- Advisors use in client consultations

Pros:

- Higher revenue per license
- Aligns with ESG/financial wellness trends

Cons:

- Changes target customer
- Potential conflicts of interest

Current Decision:

- **Keep free for now** to maximize learning and impact
 - Revisit monetization after 1,000+ users and clear product-market fit
-

15. Portfolio Narrative

What CashShield v2 Demonstrates

This project is not just a financial tool — it's a demonstration of **product thinking, technical execution, and responsible AI integration.**

1. Problem Framing & User Empathy

Skill Demonstrated: Ability to identify gaps in existing solutions and design for real user needs

Evidence:

- Identified that traditional finance tools optimize for *returns*, not *resilience*
- Recognized users need *probabilities*, not *predictions*
- Designed for "uncertain but capable" users, not crisis or wealthy segments
- Prioritized emotional safety and autonomy over prescriptive advice

Why This Matters:

- Many products are solutions in search of problems
- CashShield starts with validated pain point and builds from there

2. Iterative Product Development

Skill Demonstrated: Learning from v1, identifying structural limitations, designing v2 to address them

Evidence:

- v1 validated core hypotheses but had deployment, UX, and AI integration issues
- v2 systematically addresses each limitation without losing core value
- Explicit decision-making: why each change was necessary, what was preserved

Why This Matters:

- Shows ability to critically evaluate own work
- Demonstrates growth and responsiveness to feedback

- Highlights pragmatism (v2 isn't perfect, but it's better)

3. Responsible AI Integration

Skill Demonstrated: Using AI to augment, not replace, human judgment; designing for trust and transparency

Evidence:

- Clear separation: simulation = authoritative, AI = interpretive
- Modular architecture: AI is optional, not required
- Graceful degradation: Product works without AI
- Explicit anti-patterns: No recommendations, no financial advice
- Privacy-first: Local LLM default, transparent cloud consent

Why This Matters:

- AI hype often leads to over-reliance and opacity
- CashShield shows thoughtful, ethical AI use
- Demonstrates understanding of AI limitations and risks

4. Probabilistic Modeling & Technical Depth

Skill Demonstrated: Applying Monte Carlo simulation to consumer problem; translating complexity into simplicity

Evidence:

- Monte Carlo is typically enterprise/academic; CashShield makes it consumer-friendly
- Thoughtful parameter selection (volatility, shocks, percentiles)
- Performance optimization (vectorized NumPy, sub-3-second latency)
- Clear communication of assumptions and limitations

Why This Matters:

- Shows ability to bridge technical rigor and consumer accessibility
- Demonstrates respect for users' intelligence without overwhelming them

5. UX Psychology & Progressive Disclosure

Skill Demonstrated: Designing information architecture around human cognition and emotional needs

Evidence:

- Risk-first layout matches threat-processing psychology
- Progressive disclosure prevents overwhelm
- Emotional safety through non-judgmental language
- What-if engine transforms anxiety into agency

Why This Matters:

- Technical products often fail due to poor UX, not poor functionality
- CashShield shows awareness that UX = product, not decoration

6. Scalability & Deployment Awareness

Skill Demonstrated: Designing architecture for future growth, not just current prototype

Evidence:

- Modular AI layer enables deployment flexibility
- Streamlit for rapid iteration, with awareness of migration path
- Caching and optimization for performance at scale
- Clear roadmap from prototype → production

Why This Matters:

- Many prototypes are dead-ends; CashShield has clear path to production
 - Shows long-term thinking, not just "get it working"
-

16. Open Questions & Future Research

16.1 User Behavior

Questions:

- Do users who see high risk actually take action? What actions?
- How often do users return to re-simulate? (Quarterly? After life changes?)

- Which what-if scenarios are most commonly tested?

Why This Matters:

- Informs feature prioritization
- Validates (or challenges) theory of change: awareness → action

How to Answer:

- Analytics on what-if usage, repeat visits
- User surveys: "Did you make a change based on CashShield?"
- Longitudinal study: Track users over 6-12 months

16.2 Model Calibration

Questions:

- Are the default volatility assumptions realistic for different user segments?
- How do simulated failure rates compare to actual financial distress rates?
- Should we adjust risk thresholds (15%/35%) based on real-world data?

Why This Matters:

- Model accuracy determines trust
- If defaults are wrong, users get misleading results

How to Answer:

- Compare simulation outputs to real financial data (if available)
- User feedback: "Does this match your lived experience?"
- Academic literature on income/expense volatility by demographic

16.3 AI Explanation Quality

Questions:

- Do users prefer local (fast, private, lower quality) or cloud (slow, shared, higher quality) explanations?
- What makes an explanation "good"? (Conciseness? Analogies? Technical depth?)
- Do explanations actually improve understanding, or just satisfaction?

Why This Matters:

- Determines optimal AI strategy
- Guides prompt engineering and model selection

How to Answer:

- A/B test: Local vs. cloud explanations
- User ratings: "Was this explanation helpful?"
- Comprehension quiz: Do users understand drivers of risk better after reading explanation?

16.4 Market Positioning

Questions:

- Is "financial resilience simulator" the right framing, or is it too abstract?
- What marketing channels reach target users (anxious but capable middle-income planners)?
- Is this a B2C product, or should we pursue B2B (employers, banks)?

Why This Matters:

- Affects GTM strategy
- Determines pricing model

How to Answer:

- Landing page A/B tests
 - User acquisition experiments (Reddit, personal finance forums, LinkedIn)
 - Conversations with potential B2B partners
-

17. Conclusion

What CashShield v2 Is

CashShield v2 is a **probabilistic financial resilience simulator** that helps users understand their risk of cash insolvency, why that risk exists, and which interventions matter most.

It is:

- **Risk-aware, not risk-averse:** Acknowledges uncertainty without fear-mongering

- **Empowering, not prescriptive:** Shows users their options, trusts them to decide
- **Technical, not intimidating:** Sophisticated methodology, accessible UX
- **AI-augmented, not AI-dependent:** Uses AI to interpret, not to replace human judgment

Why It Matters

Financial fragility is pervasive but preventable. Most people operate with incomplete mental models of their risk, leading to either over-caution (foregone opportunities) or under-caution (preventable crises).

CashShield bridges the gap between "how I feel about my finances" and "what's likely to happen under realistic uncertainty."

It doesn't promise certainty. It promises **clarity**.